

# ML-Based Demand Forecasting & Inventory Planning System

- Kapeesh Tapadia

# ABSTRACT

This project presents an **ML-Based Demand Forecasting and Inventory Planning System** designed to integrate demand forecasting with inventory planning in a structured analytical workflow. The system uses historical retail sales data stored in **PostgreSQL** and processes the data through a **Python-based forecasting pipeline**. Time-series features including lag, rolling-window, and calendar-based variables are generated and used as inputs to an **XGBoost regression model** for demand forecasting at the **Store × Product Family** level.

The forecasting model is evaluated using **Mean Absolute Error (MAE)**, **Root Mean Squared Error (RMSE)**, **Mean Absolute Error Percentage (MAE%)**, and **Bias** using a time-based test approach. The implemented pipeline also generates a **30-day recursive future forecast**, with future predictions subsequently used for inventory planning. Inventory parameters including **Lead-Time Demand**, **Safety Stock**, **Reorder Point (ROP)**, and **Economic Order Quantity (EOQ)** are calculated using the generated forecasts and defined planning assumptions.

For the current V2 results stored in PostgreSQL, the system contains **26 forecast evaluation records** with an average MAE of **226.37**, RMSE of **295.51**, and MAE% of **9.49%**. The future forecasting stage produced **780 forecast records** covering **16 August 2017 to 14 September 2017**. The average inventory-planning results include a Lead-Time Demand of **16,760.66 units**, Safety Stock of **1,930.06 units**, Reorder Point of **18,690.71 units**, and EOQ of **8,215.41 units**.

The resulting forecasts, evaluation metrics, and inventory-planning outputs are stored in PostgreSQL and visualized through **Power BI**, providing an integrated workflow from historical demand data to forecasting and inventory analysis.

## 1. Introduction

Supply chain management requires effective coordination between demand, inventory, procurement, and replenishment decisions. Among these activities, demand forecasting plays an important role because future demand is uncertain and inaccurate estimates can affect inventory availability and inventory holding requirements. A forecasting system can therefore support better planning by estimating future demand from historical sales patterns.

This project presents an ML-Based Demand Forecasting & Inventory Planning System developed to combine demand forecasting with inventory planning in a structured data pipeline. The system uses historical retail sales data stored in PostgreSQL, performs data preparation and time-series feature engineering using Python, and applies the XGBoost machine learning algorithm to forecast product demand. The forecasting process uses historical demand patterns through lag and rolling-window features along with calendar-related variables.

The system was developed at the Store  $\times$  Product Family level. In the completed V2 implementation, forecasting was performed across multiple Store  $\times$  Family combinations, followed by evaluation using forecast error metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and MAE percentage. A 30-day future forecast was also generated for each processed combination.

The forecast results were subsequently used for inventory planning. Lead-Time Demand, Safety Stock, Reorder Point, and Economic Order Quantity (EOQ) were calculated using the forecast and defined inventory-planning assumptions. The resulting forecasts, evaluation metrics, and inventory-planning outputs were written back to PostgreSQL and connected to Power BI for visualization and decision-support analysis.

The overall objective of the project is therefore not only to build a demand forecasting model, but to demonstrate an integrated workflow connecting machine learning, database management, inventory planning, and business intelligence within a single analytical system.

## 2. Problem Statement

Traditional inventory planning depends heavily on historical demand information and predefined inventory policies. However, demand can vary across stores, product families, seasons, and promotional periods. Using only historical averages or fixed estimates may not adequately represent these variations, making it difficult to determine appropriate future demand and inventory requirements.

The project addresses the problem of developing an integrated analytical system that can use historical sales data to forecast future demand at the Store  $\times$  Product Family level and subsequently translate these forecasts into inventory-planning parameters. The system must also provide a structured method for evaluating forecast performance and storing the resulting outputs in a database for further analysis.

A further requirement is to connect the forecasting and inventory-planning processes with a database and visualization layer rather than treating the machine-learning model as an isolated analysis. Therefore, the developed system combines PostgreSQL for data storage and output management, Python and XGBoost for demand forecasting, inventory-planning calculations for replenishment decisions, and Power BI for visualization and analytical interpretation.

The problem is therefore defined as the development and validation of a data-driven demand forecasting and inventory-planning workflow that can process historical retail sales data, generate short-term demand forecasts, evaluate their performance, calculate inventory-planning parameters, and present the resulting information through an integrated database and dashboard architecture.

### 3. Objectives

The main objectives of the project are:

1. To develop a structured PostgreSQL database architecture for storing historical sales data and managing forecasting and inventory-planning outputs.
2. To prepare historical sales data at the Store  $\times$  Product Family level for machine-learning-based demand forecasting.
3. To develop time-series features, including lag variables, rolling-window statistics, and calendar-based features, for demand prediction.
4. To implement an XGBoost-based demand forecasting model and evaluate its performance using MAE, RMSE, MAE%, and Bias.
5. To generate a 30-day future demand forecast for the selected Store  $\times$  Product Family combinations.
6. To calculate inventory-planning parameters including Lead-Time Demand, Safety Stock, Reorder Point, and Economic Order Quantity (EOQ) using the generated forecasts and defined planning assumptions.
7. To write forecasting, evaluation, and inventory-planning outputs back to PostgreSQL using run-based output tracking.
8. To develop a Power BI dashboard for visualizing historical demand, forecasts, model performance, and inventory-planning information.

## 4. Dataset and Data Source

The project uses the Store Sales dataset, a historical retail sales dataset containing daily sales information for multiple stores and product families. The dataset provides the historical demand information required for developing and evaluating the demand forecasting model.

| Dataset Characteristic | Value                         |
|------------------------|-------------------------------|
| Total Records          | 3,000,888                     |
| Number of Stores       | 54                            |
| Product Families       | 33                            |
| Data Start Date        | 1 January 2013                |
| Data End Date          | 15 August 2017                |
| Forecasting Level      | Store $\times$ Product Family |
| Source                 | Store Sales dataset           |

The dataset contains 3,000,888 records covering the period from 1 January 2013 to 15 August 2017. It includes data from 54 stores and 33 product families. The primary sales-related variable used for forecasting is the daily sales quantity, while additional variables such as store information, product family, promotion status, and date-related information are used during data preparation and feature engineering.

For the completed project, the data was processed at the Store  $\times$  Product Family level. Rather than training a single aggregate model over the complete dataset, individual demand series were prepared for selected combinations of stores and product families. This structure allows the forecasting process to capture differences in demand behaviour between different store and product-family combinations.

The historical sales data was stored in PostgreSQL as the primary data layer of the system. Python was then used to retrieve and prepare the required data for forecasting. After model evaluation and future forecasting, the resulting forecasts and inventory-planning outputs were stored back in PostgreSQL for downstream analysis and visualization in Power BI.

```
import pandas as pd

train = pd.read_csv("../data/store_sales/train.csv")

train.head()
```

[1] Python

|   | id | date       | store_nbr | family     | sales | onpromotion |
|---|----|------------|-----------|------------|-------|-------------|
| 0 | 0  | 2013-01-01 | 1         | AUTOMOTIVE | 0.0   | 0           |
| 1 | 1  | 2013-01-01 | 1         | BABY CARE  | 0.0   | 0           |
| 2 | 2  | 2013-01-01 | 1         | BEAUTY     | 0.0   | 0           |
| 3 | 3  | 2013-01-01 | 1         | BEVERAGES  | 0.0   | 0           |
| 4 | 4  | 2013-01-01 | 1         | BOOKS      | 0.0   | 0           |

Generate Code Markdown

```
print("Stores:", train["store_nbr"].nunique())
print("Product categories:", train["family"].nunique())
print("Date range:", train["date"].min(), "to", train["date"].max())
```

[4] Python

Stores: 54  
Product categories: 33  
Date range: 2013-01-01 00:00:00 to 2017-08-15 00:00:00

```
Shape: (3000888, 6)

Columns:
['id', 'date', 'store_nbr', 'family', 'sales', 'onpromotion']

Missing values:
id          0
date        0
store_nbr   0
family      0
sales       0
onpromotion 0
dtype: int64

Data types:
id          int64
date        str
store_nbr   int64
family      str
sales       float64
onpromotion int64
dtype: object
```

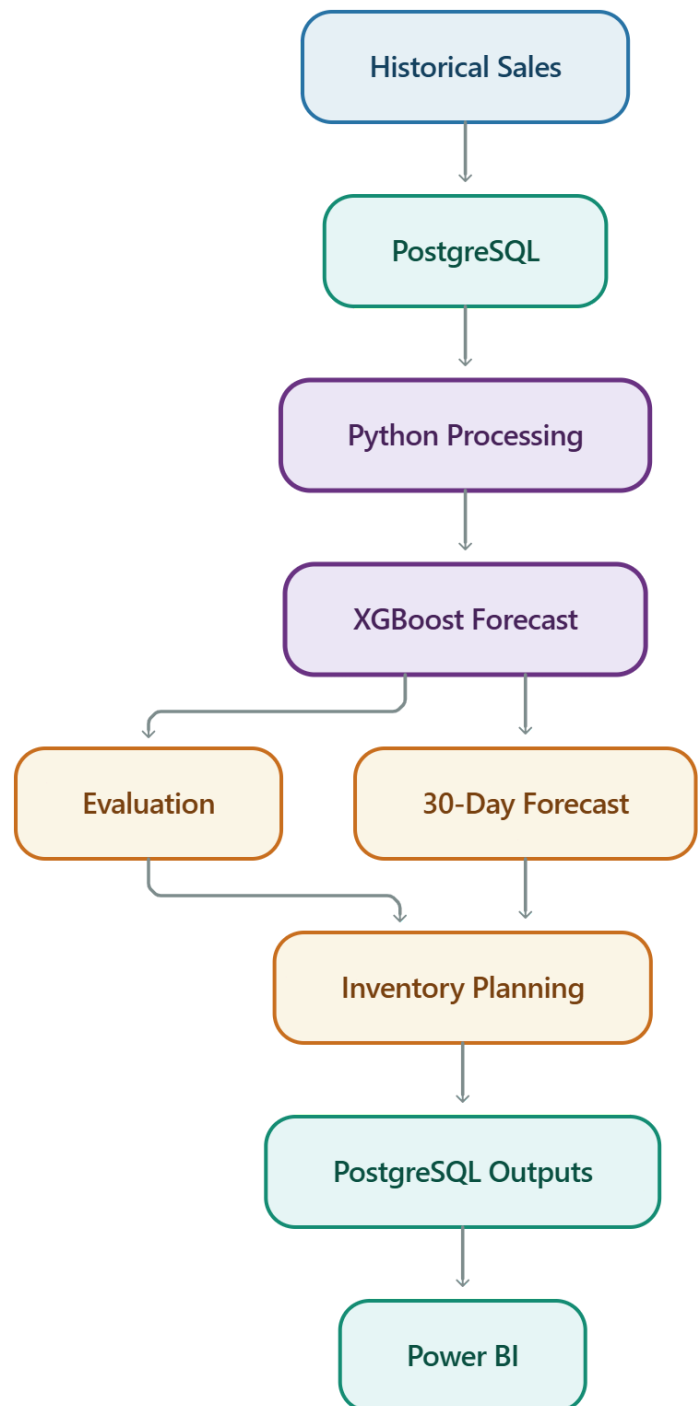
## 5. System Architecture

The developed system follows an integrated data and analytics architecture in which historical sales data is stored in PostgreSQL, processed through a Python-based forecasting pipeline, and converted into demand forecasts and inventory-planning outputs. These outputs are subsequently stored back in PostgreSQL and connected to Power BI for visualization and analysis.

The overall workflow begins with historical sales data stored in the PostgreSQL database. Python retrieves the required Store  $\times$  Product Family data and performs data preparation and feature engineering. Time-series features such as lag values, rolling statistics, and calendar variables are generated before training the forecasting model.

The XGBoost model is then used to evaluate historical demand and generate future demand forecasts. Forecast performance is evaluated using MAE, RMSE, MAE%, and Bias. After forecasting, the generated demand estimates are used for inventory-planning calculations, including Lead-Time Demand, Safety Stock, Reorder Point, and Economic Order Quantity.

The resulting forecasts, evaluation metrics, and inventory-planning outputs are written back to PostgreSQL. The database therefore acts as the central storage layer connecting the forecasting pipeline with the visualization layer. Power BI uses these stored outputs to provide interactive views of historical demand, forecasts, model performance, and inventory-planning information.



## 6. Database Design

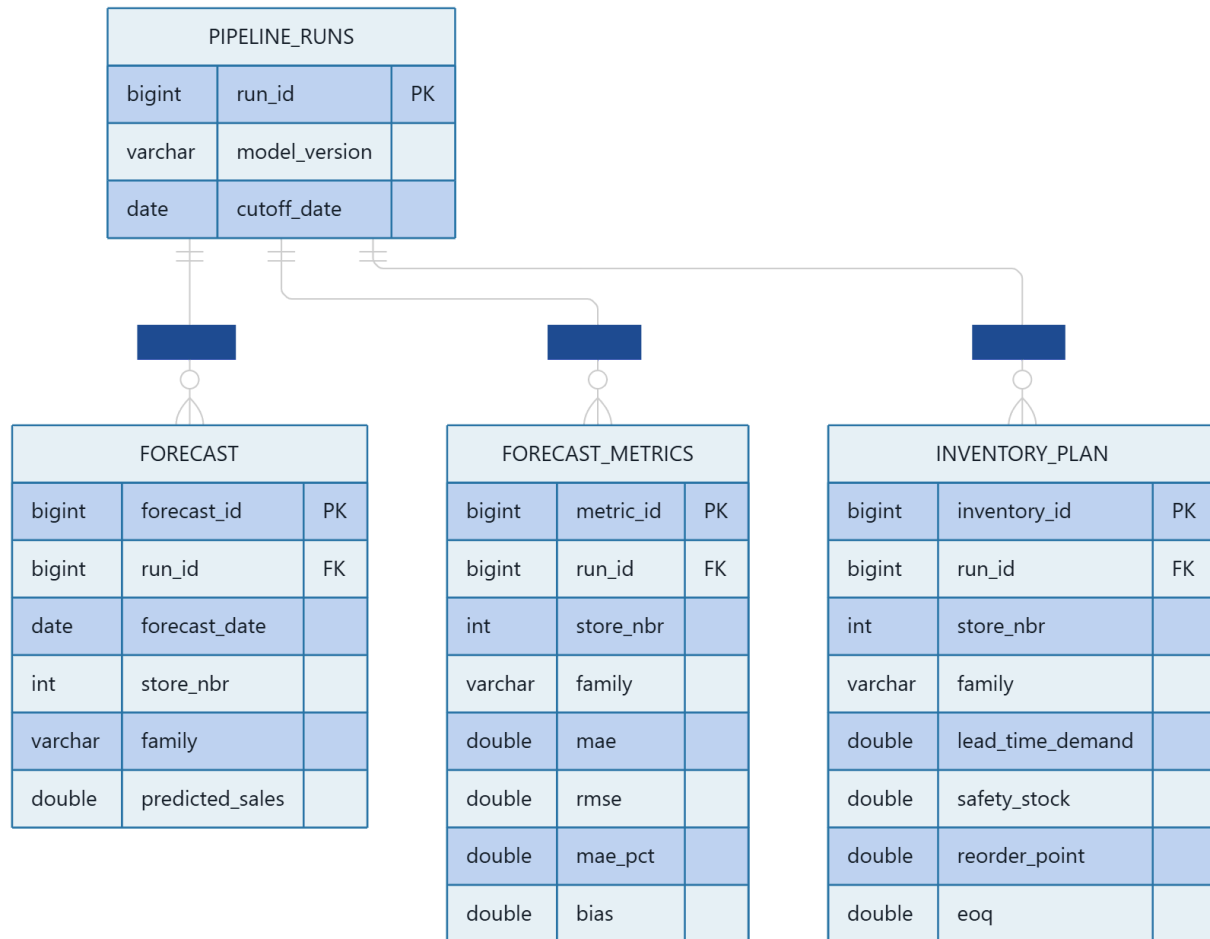
PostgreSQL was used as the central data storage layer of the system. The database stores the historical sales data used for forecasting as well as the outputs generated by the forecasting and inventory planning stages. This provides a structured and persistent data layer between the Python processing pipeline and the Power BI dashboard.

The database was organized into separate tables for historical sales, forecasting runs, forecasts, forecast evaluation metrics, and inventory planning results. A run-based structure was used to associate generated forecasts and analytical outputs with their corresponding forecasting execution.

The major tables used in the system are summarized below.

| Table                         | Purpose                      | Key Fields                                                                                                                                                                     |
|-------------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sales</code>            | Historical sales data        | <code>date</code> , <code>store_nbr</code> , <code>family</code> , <code>sales</code>                                                                                          |
| <code>pipeline_runs</code>    | Forecasting run tracking     | <code>run_id</code> , <code>model_version</code> , <code>cutoff_date</code>                                                                                                    |
| <code>forecast</code>         | Generated demand forecasts   | <code>run_id</code> , <code>forecast_date</code> , <code>store_nbr</code> , <code>family</code> , <code>predicted_sales</code>                                                 |
| <code>forecast_metrics</code> | Forecast performance metrics | <code>run_id</code> , <code>store_nbr</code> , <code>family</code> , <code>mae</code> , <code>rmse</code> , <code>mae_pct</code> , <code>bias</code>                           |
| <code>inventory_plan</code>   | Inventory planning outputs   | <code>run_id</code> , <code>store_nbr</code> , <code>family</code> , <code>lead_time_demand</code> , <code>safety_stock</code> , <code>reorder_point</code> , <code>eoq</code> |

*The `sales` table contains the historical demand data, while `pipeline_runs` provides run-level tracking for forecasting executions. The `forecast` and `forecast_metrics` tables store generated forecasts and their evaluation results, respectively. The `inventory_plan` table stores the inventory parameters calculated from the forecasting outputs.*



## 7. Data Preparation

The historical sales data was prepared before applying the forecasting model. The data was organized at the Store  $\times$  Product Family level so that demand patterns could be analyzed separately for each store and product category.

The preparation process included handling the date field, organizing observations chronologically, and preparing the sales series required for time-series feature generation. The processed data was then used as the input for feature engineering and subsequent XGBoost forecasting.

The prepared dataset provided a consistent structure for generating lag-based and rolling-window features while preserving the temporal order of the observations.

## 8. Feature Engineering

Feature engineering was performed to convert the historical sales time series into a structured feature set suitable for XGBoost forecasting. The features were designed to capture recent demand patterns, short-term trends, and calendar-related effects.

The main features used in the forecasting pipeline include lag features and rolling-window features. Lag features provide information about previous demand observations, while rolling features summarize recent demand behaviour over defined time windows. Calendar-based features were also incorporated to represent temporal patterns such as day and month effects.

These engineered features were generated separately for each Store  $\times$  Product Family combination while maintaining the chronological order of the data. The resulting feature set was then used as input to the XGBoost forecasting model.

| Feature Type      | Examples                     | Purpose                             |
|-------------------|------------------------------|-------------------------------------|
| Lag Features      | Lag 1, Lag 7, Lag 14, Lag 28 | Previous demand patterns            |
| Rolling Features  | Rolling Mean, Rolling Std    | Recent demand trend and variability |
| Calendar Features | Day, Week, Month             | Capture time-based patterns         |
| Store & Family    | Store number, Product Family | Represent forecasting level         |

```
# Calendar features
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["dayofweek"] = df["date"].dt.dayofweek
df["dayofyear"] = df["date"].dt.dayofyear
df["weekofyear"] = df["date"].dt.isocalendar().week.astype(int)

# Lag features
df["lag_1"] = df["sales"].shift(1)
df["lag_7"] = df["sales"].shift(7)
df["lag_14"] = df["sales"].shift(14)
df["lag_28"] = df["sales"].shift(28)

# Rolling features
df["rolling_mean_7"] = df["sales"].shift(1).rolling(7).mean()
df["rolling_mean_14"] = df["sales"].shift(1).rolling(14).mean()
df["rolling_mean_28"] = df["sales"].shift(1).rolling(28).mean()
```

## 9. Forecasting Methodology

XGBoost was used as the machine learning model for demand forecasting. The forecasting problem was formulated as a supervised regression task, where historical demand and engineered time-series features were used to predict future sales.

The model was trained using a time-based train-test split, with the final 30 days of the available historical period reserved for testing. This preserves the chronological order of the data and avoids using future observations during model training.

The XGBoost model was configured with 500 estimators, a learning rate of 0.05, a maximum tree depth of 6, and subsampling parameters of 0.8. The model generated predictions for the test period, which were subsequently evaluated using MAE and RMSE.

| Parameter            | Value             |
|----------------------|-------------------|
| Model                | XGBoost Regressor |
| Number of Estimators | 500               |
| Learning Rate        | 0.05              |
| Maximum Depth        | 6                 |
| Subsample            | 0.8               |
| Objective            | Regression        |

```
# XGBoost
model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42
)
```

## 10. Model Evaluation

The performance of the demand forecasting model was evaluated using a time-based test set. The final 30 days of the prepared time series were reserved for testing, while the preceding observations were used for model training. This approach preserves the chronological structure of the demand data and avoids randomly mixing future observations with the training data.

Four evaluation metrics were used to assess forecasting performance: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Error Percentage (MAE%), and Bias.

**Mean Absolute Error (MAE)** measures the average absolute difference between actual and predicted sales. A lower MAE indicates that the forecasts are, on average, closer to the observed demand values.

**Root Mean Squared Error (RMSE)** measures the square root of the average squared prediction error. Since larger errors receive greater weight, RMSE is useful for identifying cases where the model produces relatively large forecasting deviations.

**Mean Absolute Error Percentage (MAE%)** expresses the MAE relative to the average actual sales during the test period. In the implemented pipeline, it is calculated using the average actual sales of the test set. This provides a scale-independent measure that helps compare forecasting error across Store × Product Family combinations with different demand levels.

**Bias** is used to examine the direction of forecasting error, indicating whether the model tends to systematically over-predict or under-predict demand.

The calculated evaluation metrics are associated with the corresponding **run\_id**, Store, and Product Family and are stored in the PostgreSQL **forecast\_metrics** table. This allows model performance to be retained alongside the generated forecasts and linked to the specific forecasting run.

| Metric | Purpose                                                                | Interpretation                                  |
|--------|------------------------------------------------------------------------|-------------------------------------------------|
| MAE    | Measures average absolute prediction error                             | Lower values indicate smaller average errors    |
| RMSE   | Measures prediction error while giving greater weight to larger errors | Lower values indicate fewer large deviations    |
| MAE%   | Expresses MAE relative to average actual sales                         | Lower percentage indicates lower relative error |
| Bias   | Indicates the direction of forecasting error                           | Shows tendency toward over- or under-prediction |

This evaluation process provides a quantitative basis for assessing the forecasting model on previously unseen demand observations before using the model for future forecasting and inventory planning.

## 11. Future Forecasting

After evaluating the forecasting model, the trained XGBoost model was used to generate a 30-day future demand forecast. The forecasting period begins on the day immediately following the last available historical observation.

The project uses a **recursive forecasting approach**. For each future date, the model generates a prediction using calendar features, lag features, rolling demand features, and the promotion input. After a prediction is generated, it is appended to the historical series and becomes available as an input for subsequent forecasting steps. This allows the model to generate multiple future observations sequentially rather than predicting all future days independently.

For future dates, calendar variables such as year, month, day, day of week, day of year, and week of year are generated from the forecast date. Lag and rolling features are calculated from the available demand history, which increasingly includes previously generated predictions as the forecast progresses. The future **onpromotion** value is set to zero because future promotion information is not available in the implemented pipeline. Predictions are also constrained to non-negative values because negative sales quantities are not meaningful.

The forecasting process produces a dataset containing the forecast date and predicted sales for each future day. In the implemented workflow, the forecast covers **30 consecutive days** beginning immediately after the historical cutoff. The generated forecast is subsequently used as an input to the inventory-planning stage.

| Forecasting Component  | Implementation                                           |
|------------------------|----------------------------------------------------------|
| Forecast Horizon       | 30 days                                                  |
| Forecasting Method     | Recursive forecasting                                    |
| Starting Date          | Day after the last historical observation                |
| Calendar Inputs        | Year, month, day, day of week, day of year, week of year |
| Demand Inputs          | Lag and rolling-mean features                            |
| Future Promotion Input | Set to 0                                                 |
| Prediction Constraint  | Negative predictions set to 0                            |
| Output                 | Forecast date and predicted sales                        |

The recursive approach enables the system to extend the demand series beyond the historical data period while maintaining the same feature structure used during model training.

```
# Recursive 30-day forecast
forecast_days = 30
history = model_df[["date", "sales", "onpromotion"]].copy()
future_dates = pd.date_range(
    start=history["date"].max() + pd.Timedelta(days=1),
    periods=forecast_days,
    freq="D"
)
future_predictions = []
for forecast_date in future_dates:
    temp = history.copy()
    # Calendar features
    year = forecast_date.year
    month = forecast_date.month
    day = forecast_date.day
    dayofweek = forecast_date.dayofweek
    dayofyear = forecast_date.dayofyear
    weekofyear = forecast_date.isocalendar().week
    # Lag features
    lag_1 = temp["sales"].iloc[-1]
```

```

lag_7 = temp["sales"].iloc[-7]
lag_14 = temp["sales"].iloc[-14]
lag_28 = temp["sales"].iloc[-28:]
# Rolling features
rolling_mean_7 = temp["sales"].iloc[-7:].mean()
rolling_mean_14 = temp["sales"].iloc[-14:].mean()
rolling_mean_28 = temp["sales"].iloc[-28:].mean()
# Future promotion is unknown, so use 0
onpromotion = 0
X_future = pd.DataFrame([
    "onpromotion": onpromotion,
    "year": year,
    "month": month,
    "day": day,
    "dayofweek": dayofweek,
    "dayofyear": dayofyear,
    "weekofyear": int(weekofyear),
    "lag_1": lag_1,
    "lag_7": lag_7,
    "lag_14": lag_14,
    "lag_28": lag_28.iloc[-1],
    "rolling_mean_7": rolling_mean_7,
    "rolling_mean_14": rolling_mean_14,
    "rolling_mean_28": rolling_mean_28
])
prediction = final_model.predict(X_future)[0]
# Sales cannot be negative
prediction = max(0, prediction)
future_predictions.append(prediction)
# Add prediction to history for recursive forecasting
history = pd.concat([
    history,
    pd.DataFrame([
        "date": forecast_date,
        "sales": prediction,
        "onpromotion": onpromotion
    ])
], ignore_index=True)
forecast_df = pd.DataFrame({
    "forecast_date": future_dates,
    "predicted_sales": future_predictions
})

```

## 12. Inventory Planning

The demand forecasts generated by the XGBoost model were further used to derive inventory-planning parameters. The objective of this stage was to translate predicted demand into quantities that can support replenishment and stock-level planning.

The implemented inventory-planning calculations include **Lead-Time Demand, Safety Stock, Reorder Point (ROP), and Economic Order Quantity (EOQ)**. These parameters are calculated for each Store × Product Family combination using the generated forecast and the inventory assumptions defined in the forecasting pipeline.

### 12.1 Lead-Time Demand

Lead-Time Demand represents the expected demand during the replenishment lead-time period. It is calculated from the forecasted daily demand over the specified lead-time horizon.

This provides an estimate of the quantity that may be required while waiting for replenishment to arrive.

### 12.2 Safety Stock

Safety Stock is used as a demand-protection quantity to account for uncertainty in forecasted demand. In the implemented pipeline, safety stock is derived using the forecast-demand variability and the specified service-level and lead-time assumptions.

The resulting value provides an additional inventory quantity intended to reduce the risk of stockout when actual demand differs from the forecast.

### 12.3 Reorder Point

The Reorder Point combines expected demand during lead time with the calculated safety stock. It represents the inventory level at which replenishment should be considered.

The implemented calculation follows:

**Reorder Point = Lead-Time Demand + Safety Stock**

Thus, the reorder point accounts for both expected demand during replenishment and an additional buffer against demand uncertainty.

### 12.4 Economic Order Quantity

Economic Order Quantity (EOQ) estimates an order quantity intended to balance ordering and inventory-holding considerations. The implemented calculation uses the demand-related input together with the specified ordering-cost and holding-cost assumptions.

EOQ is therefore treated as an inventory-planning parameter rather than as a directly optimized procurement quantity for a specific real-world business.

## 12.5 Inventory Planning Output

The calculated inventory parameters are generated at the Store  $\times$  Product Family level and stored in PostgreSQL. Each inventory record is associated with its corresponding **run\_id**, allowing the inventory-planning results to be linked to the forecasting run that produced the underlying demand estimates.

| Parameter        | Purpose                                                                                    |
|------------------|--------------------------------------------------------------------------------------------|
| Lead-Time Demand | Estimates demand expected during replenishment lead time                                   |
| Safety Stock     | Provides a buffer against demand uncertainty                                               |
| Reorder Point    | Indicates the inventory level at which replenishment should be considered                  |
| EOQ              | Estimates an economically appropriate order quantity based on the defined cost assumptions |

The inventory-planning stage therefore extends the forecasting workflow from demand prediction to a set of quantitative inventory parameters that can be used for further analysis and visualization.

| Parameter                    | Value                                      | Used For                                             |
|------------------------------|--------------------------------------------|------------------------------------------------------|
| Lead Time                    | <b>7 days</b>                              | Lead-Time Demand and Safety Stock                    |
| Service Level Z-Value        | <b>1.65</b>                                | Safety Stock calculation                             |
| Ordering Cost per Order      | <b>₹500</b>                                | EOQ calculation                                      |
| Annual Holding Cost per Unit | <b>₹10</b>                                 | EOQ calculation                                      |
| Annual Demand Basis          | <b>Average annual demand for 2013–2016</b> | EOQ calculation                                      |
| 2017 Data                    | <b>Excluded</b>                            | Annual-demand calculation because 2017 is incomplete |

## 13. PostgreSQL Integration

PostgreSQL was used as the central data-storage layer for the forecasting and inventory-planning workflow. The Python forecasting pipeline connects to the database to retrieve historical sales data and to store the outputs generated during each forecasting execution.

A run-based tracking mechanism was implemented using `run_id`. Each forecasting execution is associated with a run identifier, which allows generated forecasts and analytical outputs to be linked to the corresponding execution and model version.

After model prediction, the generated forecast records are written to the `forecast` table. The evaluation stage stores the calculated MAE, RMSE, MAE%, and Bias values in the `forecast_metrics` table. The inventory-planning stage similarly stores Lead-Time Demand, Safety Stock, Reorder Point, and EOQ values in the `inventory_plan` table.

The resulting database structure separates historical input data from forecasting and analytical outputs while maintaining a relationship between the outputs and their corresponding pipeline runs.

The overall database write-back flow can be summarized as:

**Historical Sales → Python Forecasting Pipeline → Forecast Output → Evaluation Metrics → Inventory Planning → PostgreSQL**

The stored outputs provide a persistent data layer for subsequent analysis and visualization in Power BI. This integration also makes it possible to retain results from different forecasting executions rather than relying only on temporary notebook outputs.

| Output             | PostgreSQL Table              | Main Information Stored                       |
|--------------------|-------------------------------|-----------------------------------------------|
| Historical sales   | <code>sales</code>            | Date, store, product family, sales            |
| Pipeline execution | <code>pipeline_runs</code>    | Run ID, model version, cutoff date            |
| Demand forecast    | <code>forecast</code>         | Forecast date, store, family, predicted sales |
| Model evaluation   | <code>forecast_metrics</code> | MAE, RMSE, MAE%, Bias                         |
| Inventory planning | <code>inventory_plan</code>   | Lead-Time Demand, Safety Stock, ROP, EOQ      |

This database integration forms the connection between the Python-based analytical pipeline and the reporting layer used for dashboard development.

The code demonstrates the write-back of generated forecast results from Python to the PostgreSQL database. The forecast data is first prepared with the corresponding `run_id`, Store number, Product Family, forecast date, and predicted sales, and is then appended to the `forecast` table. The evaluation metrics and inventory planning outputs are also written to their respective PostgreSQL tables using the same approach. This ensures that the outputs generated by each pipeline execution are persistently stored and can be used for subsequent analysis and Power BI visualization.

```
# =====  
# WRITE FORECAST TO SQL  
# =====  
  
forecast_to_sql = forecast_df.copy()  
  
forecast_to_sql["run_id"] = run_id  
forecast_to_sql["store_nbr"] = DEV_STORE  
forecast_to_sql["family"] = DEV_FAMILY  
  
forecast_to_sql = forecast_to_sql[  
    [  
        "run_id",  
        "forecast_date",  
        "store_nbr",  
        "family",  
        "predicted_sales"  
    ]  
]  
  
forecast_to_sql.to_sql(  
    "forecast",  
    engine,  
    schema="public",  
    if_exists="append",  
    index=False,  
    method="multi"  
)
```

## 14. Power BI Dashboard

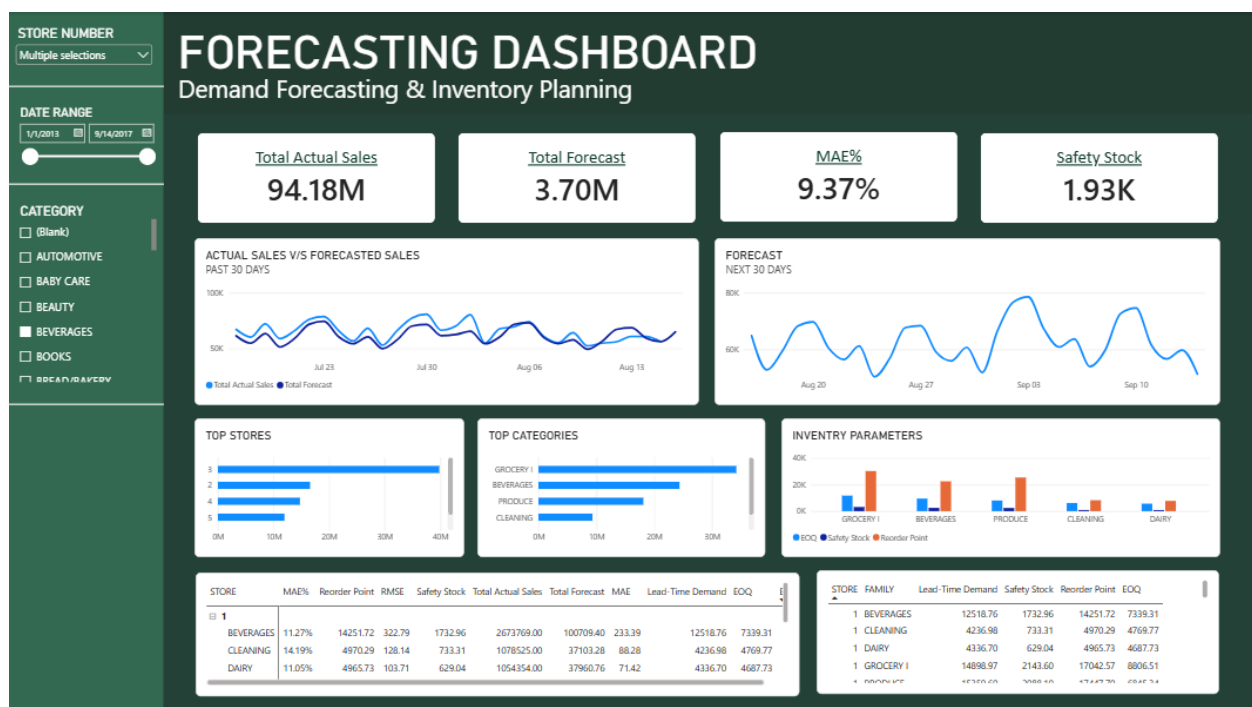
Power BI was used as the visualization and analytical layer of the system. The dashboard was connected directly to the PostgreSQL database, allowing the stored historical sales, forecast, evaluation metrics, inventory planning outputs, and pipeline-run information to be analyzed in a single interface.

The Power BI data model was designed to support Store, Product Family, and date-based filtering while maintaining the relationship between forecasts and their corresponding pipeline runs. A **DimRun** structure was used to handle multiple pipeline executions and enable selection of the latest available run for a Store  $\times$  Product Family combination.

The dashboard brings together the major analytical components of the project:

| Dashboard Component   | Purpose                                                       |
|-----------------------|---------------------------------------------------------------|
| Historical Demand     | Analyze historical sales trends                               |
| Forecast              | Visualize predicted demand                                    |
| Forecast Performance  | Review MAE, RMSE, MAE% and Bias                               |
| Store/Family Analysis | Compare demand across selected dimensions                     |
| Inventory Planning    | Present Lead-Time Demand, Safety Stock, Reorder Point and EOQ |

The dashboard therefore provides a visual interface connecting the forecasting results with their corresponding inventory-planning outputs, rather than presenting the machine-learning model as an isolated analysis.



# 15. Results and Analysis

The developed forecasting and inventory planning system was evaluated using the outputs generated by the V2 forecasting pipeline and stored in PostgreSQL. The analysis focuses on forecasting performance, future demand prediction, and the conversion of forecast results into inventory planning parameters. The results are presented at the Store  $\times$  Product Family level and are based on the currently stored V2 records.

## 15.1 Forecasting Performance

The forecasting performance varied across different Store  $\times$  Product Family combinations. The model was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Error Percentage (MAE%). These metrics provide information about the magnitude and relative size of forecasting errors.

For the current V2 results, **26 metric records** were available in PostgreSQL. The average MAE across these records was **226.37**, while the average RMSE was **295.51**. The average MAE% was **9.49%**. The Bias value was not available in the current aggregated result because the corresponding stored values were blank.

The variation across Store  $\times$  Product Family combinations indicates that the forecasting error was not uniform across all demand series. Store 3 – DAIRY produced the lowest MAE% among the evaluated combinations, while Store 5 – CLEANING had the highest MAE% among the reported higher-error combinations.

**Table 15.1: Best and Higher-Error Store × Product Family Forecasting Results**

| Performance  | Store | Product Family | MAE    | RMSE   | MAE%   |
|--------------|-------|----------------|--------|--------|--------|
| Best         | 3     | DAIRY          | 82.76  | 99.55  | 4.00%  |
| Best         | 3     | PRODUCE        | 386.24 | 456.94 | 4.91%  |
| Best         | 2     | GROCERY I      | 248.11 | 311.20 | 6.50%  |
| Best         | 5     | PRODUCE        | 105.13 | 128.18 | 6.55%  |
| Best         | 2     | PRODUCE        | 155.45 | 186.07 | 6.57%  |
| Higher Error | 5     | CLEANING       | 130.71 | 163.37 | 14.30% |
| Higher Error | 1     | CLEANING       | 88.28  | 128.14 | 14.19% |
| Higher Error | 4     | CLEANING       | 106.31 | 138.71 | 13.13% |
| Higher Error | 1     | GROCERY I      | 295.09 | 388.51 | 12.34% |
| Higher Error | 5     | BEVERAGES      | 229.35 | 332.00 | 11.97% |

The lowest MAE% in the reported results was observed for **Store 3 – DAIRY at 4.00%**, indicating relatively low prediction error for that demand series. Store 3 – PRODUCE also showed a relatively low MAE% of **4.91%**. At the higher-error end, Store 5 – CLEANING recorded an MAE% of **14.30%**, followed by Store 1 – CLEANING at **14.19%**.

These differences show that model performance depends on the individual Store × Product Family demand series. Some combinations were easier for the model to forecast accurately, whereas others produced comparatively larger prediction errors.

## 15.2 Inventory Planning Results

The future demand forecasts were subsequently used to calculate inventory planning parameters. The inventory planning stage generated values for Lead-Time Demand, Safety Stock, Reorder Point, and Economic Order Quantity (EOQ).

For the current V2 results, **26 inventory records** were available. The average Lead-Time Demand was **16,760.66 units**, while the average Safety Stock was **1,930.06 units**. The resulting average Reorder Point was **18,690.71 units**. The average EOQ was **8,215.41 units**.

**Table 15.2: Average Inventory Planning Parameters for V2 Results**

| Inventory Parameter | Average Value |
|---------------------|---------------|
| Lead-Time Demand    | 16,760.66     |
| Safety Stock        | 1,930.06      |
| Reorder Point       | 18,690.71     |
| EOQ                 | 8,215.41      |

The Lead-Time Demand represents the expected demand during the assumed replenishment lead time. Safety Stock provides an additional inventory buffer based on forecast variability and the specified service-level assumption. The Reorder Point combines these two components to indicate the inventory level at which replenishment should be considered.

EOQ provides an estimated replenishment quantity based on the demand and the specified ordering and holding cost assumptions. Therefore, the inventory planning stage extends the forecasting output beyond demand prediction and converts it into quantitative replenishment-related parameters.

The reported inventory values should be interpreted within the assumptions defined in the implementation, including the **7-day lead time, service-level Z-value of 1.65, ordering cost of ₹500 per order, and annual holding cost of ₹10 per unit**. These are project-level assumptions rather than observed business parameters.

## 15.3 Future Forecast Results

The implemented forecasting pipeline generated future demand predictions for a **30-day horizon** following the last available historical observation.

For the current V2 results stored in PostgreSQL, a total of **780 forecast records** were generated across **5 stores and 5 product families**, corresponding to **26 V2 forecasting runs**. The forecast period extends from **16 August 2017 to 14 September 2017**.

The total predicted sales across these forecast records were **1,867,615.94 units**, with an average predicted sales value of **2,394.38 units per forecast record**.

**Table 15.3: Summary of V2 Future Demand Forecast Results**

| Parameter                | Result            |
|--------------------------|-------------------|
| Total Forecast Records   | 780               |
| Stores Covered           | 5                 |
| Product Families Covered | 5                 |
| V2 Forecasting Runs      | 26                |
| Forecast Horizon         | 30 days           |
| Forecast Start Date      | 16 August 2017    |
| Forecast End Date        | 14 September 2017 |
| Total Predicted Sales    | 1,867,615.94      |
| Average Predicted Sales  | 2,394.38          |

The forecast record count is consistent with the 30-day forecasting horizon, as **26 forecasting runs**  $\times$  **30 forecast days** = **780 forecast records**. The forecast results were stored in PostgreSQL together with their corresponding **run\_id**, Store, Product Family, forecast date, and predicted sales value.

The latest V2 run was **run\_id 103**, corresponding to **Store 5 – PRODUCE**. This run generated 30 daily forecasts from 16 August 2017 to 14 September 2017, with a total predicted demand of **43,535.42 units** and an average daily prediction of **1,451.18 units**.

## 15.4 Overall Analysis

The results demonstrate the complete flow from demand forecasting to inventory planning implemented in the project. The XGBoost model generated demand forecasts using the prepared time-series features, and its performance was evaluated using multiple error metrics. The average MAE% of **9.49%** across the current V2 metric records provides an overall indication of the relative forecasting error represented in the stored evaluation results.

The Store  $\times$  Product Family analysis also shows that forecasting performance varies across demand series. The difference between the lower-error and higher-error combinations indicates that the same forecasting approach does not produce identical error levels for every store and product family.

The future forecasting stage extended the model output to a 30-day period, generating **780 forecast records** across the currently stored V2 forecasting runs. These predictions were then used in the inventory planning stage to calculate Lead-Time Demand, Safety Stock, Reorder Point, and EOQ.

The integration of these stages provides a continuous analytical workflow:

**Historical Sales → Forecasting → Forecast Evaluation → Future Demand Forecast → Inventory Planning → PostgreSQL Storage → Power BI Visualization**

Overall, the implemented system demonstrates how machine learning-based demand forecasting can be connected with inventory planning calculations and a database-backed analytical layer. The results presented here represent the outputs of the implemented project pipeline and should be interpreted within the dataset scope and inventory assumptions used in the system.

# 16. Limitations

Although the developed system successfully integrates demand forecasting, forecast evaluation, inventory planning, PostgreSQL storage, and Power BI visualization, several limitations should be considered when interpreting the results.

## 16.1 Dataset Limitations

The forecasting results are based on the historical Store Sales dataset used in the project. The analysis was performed at the **Store × Product Family** level for the combinations processed by the implemented pipeline. Therefore, the reported forecasting performance represents the evaluated data scope and should not be interpreted as a general measure of forecasting performance for all possible stores, products, or retail environments.

The dataset provides historical sales information, but real-world demand may also be affected by additional factors such as changing market conditions, external events, pricing decisions, supply constraints, and other business variables that are not represented in the implemented forecasting workflow.

## 16.2 Forecasting Limitations

The forecasting model uses historical demand patterns together with lag, rolling, and calendar-based features. Although these features capture recent demand behaviour and time-related patterns, they may not fully represent sudden or unexpected changes in demand.

Forecasting performance also varies across Store × Product Family combinations. In the evaluated V2 results, MAE% ranged from **4.00%** for Store 3 – DAIRY among the reported lower-error combinations to **14.30%** for Store 5 – CLEANING among the reported higher-error combinations. This indicates that model performance is dependent on the characteristics of the individual demand series.

The future forecasting stage also uses a recursive forecasting approach, where predictions generated for future dates become inputs for subsequent predictions. Consequently, forecasting errors at an earlier future step may propagate to later forecast steps.

Another limitation is the treatment of future promotion information. Since future promotion values were not available for the forecasting horizon, the implementation used **0** for the future **onpromotion** input. Actual future promotional activity could therefore differ from the assumption used during forecasting.

## 16.3 Inventory Planning Assumptions

The inventory planning calculations depend on predefined project assumptions rather than actual business-specific operating parameters.

The implemented calculations use a **7-day lead time** and a **service-level Z-value of 1.65** for Lead-Time Demand, Safety Stock, and Reorder Point calculations. EOQ is calculated using an ordering cost of **₹500 per order** and an annual holding cost of **₹10 per unit**.

The annual demand used for EOQ is based on the average annual demand from the complete years **2013–2016**, while the incomplete 2017 year is excluded from this calculation.

These assumptions are suitable for demonstrating the integration of forecasting with inventory planning, but actual deployment would require organization-specific values for supplier lead times, service-level targets, ordering costs, holding costs, and other inventory policies.

## 16.4 Implementation and Validation Limitations

The current implementation demonstrates an integrated analytical workflow, but it should not be considered a production deployment. The system was developed and evaluated within the scope of the project using the available dataset, implemented forecasting pipeline, PostgreSQL database, and Power BI dashboard.

The current pipeline also creates `run_id` values at the execution level associated with individual Store × Product Family processing rather than representing one single portfolio-level pipeline run. This structure is retained in the current implementation and provides run-based traceability for the stored outputs, but it could be further refined in a future version.

The project also evaluates model performance using historical holdout data. Although this provides a time-based evaluation of the forecasting model, actual operational performance such as stockout reduction, inventory-cost reduction, service-level improvement, or financial savings has not been measured. Therefore, no direct business-impact claims are made based on the current results.

Overall, these limitations define the scope within which the reported results should be interpreted. The system demonstrates the technical integration of machine learning forecasting, inventory planning, database storage, and visualization, while further validation with real operational parameters and additional external demand factors would be required for practical deployment.

# 17. Future Scope

The developed system provides an integrated foundation for demand forecasting and inventory planning. The following extensions can further improve the capability and practical applicability of the system.

## 17.1 Advanced Forecasting Models

The forecasting pipeline can be extended by evaluating additional machine learning and time-series models such as LightGBM, Random Forest, Prophet, or other suitable forecasting approaches. Model comparison across Store  $\times$  Product Family combinations could help identify the most suitable forecasting method for different demand patterns.

## 17.2 Additional Demand Drivers

Additional variables such as pricing, holidays, promotions, weather conditions, local events, and other relevant external factors can be incorporated into the feature engineering process. Including these variables could help the model capture demand changes that cannot be explained only through historical sales and calendar features.

## 17.3 Improved Promotion Forecasting

The current implementation uses a value of zero for future promotion information because actual future promotion inputs are unavailable. A future version could incorporate planned promotional schedules or a separate mechanism for estimating promotional effects before generating future forecasts.

## 17.4 Dynamic Inventory Parameters

The current inventory planning stage uses predefined lead-time, service-level, ordering-cost, and holding-cost assumptions. Future development could allow these parameters to vary by Store  $\times$  Product Family combination based on actual operational characteristics. This would make the inventory recommendations more closely aligned with individual replenishment requirements.

## 17.5 Automated Pipeline Execution

The current workflow can be extended into a scheduled pipeline in which data extraction, feature generation, model execution, forecast generation, inventory planning, and PostgreSQL write-back are executed automatically at predefined intervals. This could reduce manual execution and provide a more repeatable forecasting workflow.

## 17.6 Model Monitoring and Retraining

A future implementation could continuously monitor forecasting performance using stored evaluation metrics. Significant deterioration in model performance could be used as a trigger for model retraining or parameter adjustment. This would help maintain forecasting quality as demand patterns change over time.

## 17.7 Enhanced Power BI Analytics

The Power BI dashboard can be extended with additional analytical features such as historical-versus-forecast comparisons, Store  $\times$  Product Family drill-downs, forecast-error analysis, inventory-risk indicators, and trend-based filtering. This could provide users with more detailed analytical views of the forecasting and inventory planning outputs.

## 17.8 Improved Run Management

The existing run-based architecture can be further refined so that a single pipeline execution represents one complete forecasting cycle across all required Store  $\times$  Product Family combinations. This would simplify run-level analysis and make comparison between complete forecasting cycles more straightforward.

## 17.9 Deployment and Operational Integration

After additional validation, the system could be integrated with operational inventory or enterprise systems. Such an extension could allow forecasting and inventory recommendations to become part of a broader decision-support workflow. Before such deployment, the model would require validation using real operational parameters, additional demand drivers, and appropriate business performance measures.

### Overall Future Scope

The future development of the system can therefore focus on three major areas: **improving forecast quality, making inventory planning more adaptive, and increasing automation and operational usability**. These extensions would build upon the current PostgreSQL–Python–XGBoost–Inventory Planning–Power BI architecture rather than requiring a complete redesign of the system.

# 18. Conclusion

The project successfully developed an integrated **ML-Based Demand Forecasting and Inventory Planning System** that combines machine learning, database management, inventory planning, and business intelligence into a single analytical workflow.

The system uses historical sales data stored in PostgreSQL and processes the data through a Python-based forecasting pipeline. Historical demand is transformed into time-series features including lag, rolling, and calendar-based variables, which are then used by an **XGBoost regression model** to generate demand forecasts at the Store  $\times$  Product Family level.

The forecasting stage was evaluated using MAE, RMSE, MAE%, and Bias. The current V2 results stored in PostgreSQL contained **26 evaluation records**, with an average MAE of **226.37**, average RMSE of **295.51**, and average MAE% of **9.49%**. The results also showed variation in forecasting performance across different Store  $\times$  Product Family combinations, demonstrating the importance of evaluating individual demand series rather than relying only on an overall model measure.

The system further generated **30-day future demand forecasts**. The current V2 outputs contained **780 forecast records** across 26 forecasting runs, covering the period from **16 August 2017 to 14 September 2017**. These forecast outputs were subsequently used to derive inventory planning parameters.

The inventory planning component calculated **Lead-Time Demand, Safety Stock, Reorder Point, and Economic Order Quantity (EOQ)** using the assumptions defined in the implementation. For the current V2 inventory results, the average Lead-Time Demand was **16,760.66 units**, average Safety Stock was **1,930.06 units**, average Reorder Point was **18,690.71 units**, and average EOQ was **8,215.41 units**.

PostgreSQL acts as the central data storage layer, maintaining historical sales and generated analytical outputs while using **run\_id** to provide traceability between forecasting, evaluation, and inventory planning results. The stored outputs are subsequently made available through Power BI, providing a visual analytical interface for examining historical demand, forecasts, model performance, and inventory planning parameters.

Overall, the project demonstrates the implementation of an end-to-end analytical workflow in which **historical demand data is transformed into machine learning forecasts and subsequently converted into inventory planning information**. The integration of PostgreSQL, Python, XGBoost, inventory planning calculations, and Power BI provides a structured foundation for demand-driven inventory analysis.

The developed system is intended as an academic and technical implementation rather than a production deployment. Its results are therefore interpreted within the scope of the available dataset, implemented forecasting methodology, and predefined inventory assumptions. Further improvements such as additional demand drivers, dynamic inventory parameters, model monitoring, automated execution, and operational validation can extend the system toward more practical applications.

